

Object-Oriented Design Document

Todo List Course Project

CS 3354.253 | Java Desktop GUI Application

Package: txst.myApp | Java 21 · Gradle · Swing · Text-File Storage

1. Project Description	3
1.1 Application Overview	3
1.2 Technology Stack	3
2. CRC Cards	4
2.1 Task	4
2.2 TaskManager	4
2.3 TaskFileStorage	4
2.4 TaskService	5
2.5 TodoFrame	5
2.6 TaskDialog	5
3. Individual UML Class Diagrams	7
3.1 App	7
3.2 Task	7
3.3 TaskManager	7
3.4 TaskFileStorage	8
3.5 TaskService	8
3.6 TodoFrame	9
3.7 TaskDialog (with Nested Record)	9
4. Class Descriptions and Business Rules	11
4.1 Class Descriptions	11
App	11
Task	11
TaskManager	11
TaskFileStorage	11
TaskService	11
TodoFrame	12
TaskDialog	12
4.2 Business Rules	12
5. Class Relationship Diagram	13
5.1 Relationship Overview	13
5.2 Relationship Descriptions	13
5.3 Architectural Summary	14

1. Project Description

The Todo List Course Project is a Java desktop application built for CS 3354. It provides users with a graphical interface for managing personal tasks through a Swing-based GUI. The application follows a layered architecture separating the user interface, business logic, and file persistence responsibilities into distinct classes.

1.1 Application Overview

Users interact with the application through a main window (TodoFrame) that displays a table of tasks. From this window, users may perform the following operations:

1. Create a new task with a title and optional description.
2. Update the title or description of an existing task.
3. Mark a task as completed.
4. Delete a task after user confirmation.
5. Display all tasks or filter to show only completed tasks.
6. Save and load task data automatically using a plain text file.

1.2 Technology Stack

Component	Detail
Language	Java 21
Build Tool	Gradle
GUI Framework	Java Swing
Data Storage	Plain text file (data/tasks.txt)
Package	txst.myApp
Record format	id title description completed

2. CRC Cards

CRC (Class–Responsibility–Collaborator) cards summarize each class's role in the system. The left column lists what the class is responsible for doing; the right column lists the classes it depends on or interacts with.

2.1 Task

Class: Task	
Responsibilities	Collaborators
Store task data (id, title, description, completed). Provide accessor and mutator methods for all fields. Represent task completion status via a boolean flag. Return a human-readable string through toString().	TaskManager (creates and manages Task instances) TaskFileStorage (serializes/deserializes Task objects) TaskService (passes Task references to GUI) TodoFrame (displays Task data in table) TaskDialog (prefills form fields from a Task)

2.2 TaskManager

Class: TaskManager	
Responsibilities	Collaborators
Maintain the in-memory list of Task objects. Add a new task and assign it a generated integer ID. Update the title and description of an existing task. Mark a task as completed by ID. Delete a task by ID from the list. Return all tasks or only completed tasks. Find a task by its ID using Optional. Validate that a task title is not blank. Generate the next available task ID.	Task (creates, stores, and modifies Task objects) List<Task> / ArrayList<Task> (Java Collections) Optional<Task> (returned by findTaskById)

2.3 TaskFileStorage

Class: TaskFileStorage	
Responsibilities	Collaborators
Load a list of Task objects from a text file. Save a list of Task objects to a text file.	Task (reads and writes Task fields) Path (java.nio.file.Path for file location)

Return an empty list if the file does not exist. Create parent directories before saving. Format a Task as a pipe-delimited text record. Parse a text record back into a Task object. Escape and unescape separators, backslashes, and newlines in field values.	Files (java.nio.file.Files for I/O operations) List<Task> / ArrayList<Task>
--	--

2.4 TaskService

Class: TaskService	
Responsibilities	Collaborators
Load tasks from the file when constructed. Delegate all task operations to TaskManager. Automatically save tasks after every successful mutation. Expose task query methods to the GUI layer.	TaskFileStorage (loads and saves task data) TaskManager (performs business logic) Task (returned in method results) List<Task> (returned by getAllTasks, getCompletedTasks) Optional<Task> (returned by findTaskById)

2.5 TodoFrame

Class: TodoFrame	
Responsibilities	Collaborators
Display the main application window. Render tasks in a non-editable JTable with four columns. Allow users to filter between all tasks and completed tasks. Handle user actions: add, edit, complete, delete. Open TaskDialog for add and edit operations. Refresh the table after every data change. Show informational and error dialogs via JOptionPane.	TaskService (data access and mutations) Task (read to populate table rows) TaskDialog (opened for add/edit) JTable, DefaultTableModel (Swing table display) JComboBox (filter selector) JOptionPane (dialogs) JFrame (parent window class)

2.6 TaskDialog

Class: TaskDialog	
Responsibilities	Collaborators
Display a modal dialog with title and description input fields. Prefill fields when editing an existing task.	Task (prefills fields from an existing task when editing) TodoFrame (owner window; receives TaskFormData result)

Validate that the title field is not blank. Return the entered form data wrapped in Optional to the caller. Close without saving when the user cancels.	Optional<TaskFormData> (wraps the return value) JDialog, JTextField, JTextArea, JOptionPane
--	---

3. Individual UML Class Diagrams

Each diagram follows the standard UML class notation: class name in the top compartment, attributes in the middle, and methods in the bottom. Visibility is indicated by + (public) and - (private). Static and final members are noted in comments.

3.1 App

App
<pre>- TASK_FILE_PATH : Path {static final}</pre>
<pre>+ getGreeting() : String + createTaskService() : TaskService + createTodoFrame(taskService : TaskService) : TodoFrame + start() : void + main(args : String[]) : void {static}</pre>

3.2 Task

Task
<pre>- id : int - title : String - description : String - completed : boolean</pre>
<pre>+ Task(id : int, title : String, description : String, completed : boolean) + getId() : int + setId(id : int) : void + getTitle() : String + setTitle(title : String) : void + getDescription() : String + setDescription(description : String) : void + isCompleted() : boolean + setCompleted(completed : boolean) : void + toString() : String</pre>

3.3 TaskManager

TaskManager
<pre>- tasks : List<Task> - nextTaskId : int</pre>
<pre>+ TaskManager() + TaskManager(startingTasks : List<Task>) + addTask(title : String, description : String) : Task</pre>

```

+ updateTask(id : int, title : String, description : String) : boolean
+ markTaskCompleted(id : int) : boolean
+ deleteTask(id : int) : boolean
+ getAllTasks() : List<Task>
+ getCompletedTasks() : List<Task>
+ findTaskById(id : int) : Optional<Task>
- validateTitle(title : String) : void
- cleanDescription(description : String) : String
- findNextTaskId() : int

```

3.4 TaskFileStorage

TaskFileStorage

```

- FIELD_SEPARATOR : String    {static final, value = "|"}
- filePath : Path

+ TaskFileStorage(filePath : Path)
+ loadTasks() : List<Task>
+ saveTasks(tasks : List<Task>) : void
- formatTask(task : Task) : String
- parseTask(line : String) : Task
- splitEscapedFields(line : String) : List<String>
- escape(value : String) : String
- unescape(value : String) : String

```

3.5 TaskService

TaskService

```

- storage : TaskFileStorage
- taskManager : TaskManager

+ TaskService(storage : TaskFileStorage)
+ addTask(title : String, description : String) : Task
+ updateTask(id : int, title : String, description : String) : boolean
+ markTaskCompleted(id : int) : boolean
+ deleteTask(id : int) : boolean
+ getAllTasks() : List<Task>
+ getCompletedTasks() : List<Task>
+ findTaskById(id : int) : Optional<Task>
+ saveTasks() : void

```


3.6 TodoFrame

TodoFrame
<pre> - TABLE_COLUMNS : String[] {static final, values: ID, Title, Description, Status} - ALL_TASKS_FILTER : String {static final} - COMPLETED_TASKS_FILTER : String {static final} - taskService : TaskService - tableModel : DefaultTableModel - taskTable : JTable - filterComboBox : JComboBox<String> </pre>
<pre> + TodoFrame(taskService : TaskService) - createTableModel() : DefaultTableModel - createTaskTable() : JTable - createFilterComboBox() : JComboBox<String> - createHeaderPanel() : JPanel - createActionPanel() : JPanel - handleAddTask(event : ActionEvent) : void - handleEditTask(event : ActionEvent) : void - refreshTaskTable() : void - getVisibleTasks() : List<Task> - handleCompleteTask(event : ActionEvent) : void - handleDeleteTask(event : ActionEvent) : void - getSelectedTaskId() : Optional<Integer> - showMessage(message : String) : void - showError(message : String, exception : IOException) : void </pre>

3.7 TaskDialog (with Nested Record)

TaskDialog
<pre> - titleField : JTextField - descriptionArea : JTextArea - taskFormData : TaskFormData </pre>
<pre> + TaskDialog(owner : Frame, dialogTitle : String, existingTask : Task) + showForCreate(owner : Frame) : Optional<TaskFormData> {static} + showForEdit(owner : Frame, task : Task) : Optional<TaskFormData> {static} - createFormPanel() : JPanel - createButtonPanel() : JPanel - saveTaskData() : void </pre>

Nested Record: TaskDialog.TaskFormData

<<record>> TaskDialog.TaskFormData
<pre> + title : String + description : String </pre>

```
+ TaskFormData(title : String, description : String)
+ title() : String
+ description() : String
```

4. Class Descriptions and Business Rules

4.1 Class Descriptions

App

App is the entry point of the application. It initializes the TaskFileStorage, constructs a TaskService, and creates the TodoFrame. It launches the Swing event dispatch thread using SwingUtilities.invokeLater(). If the task file cannot be loaded at startup, it presents an error dialog using JOptionPane and exits gracefully.

Task

Task is the core domain entity. It stores the four fields that define a task record: an integer identifier, a title string, a description string, and a boolean completion flag. All fields are accessible through standard getter and setter methods. The toString() method returns the task title, which is used for display purposes in certain contexts.

TaskManager

TaskManager owns the in-memory list of Task objects and contains all business logic for task operations. It is constructed either empty or from an existing list of tasks loaded from storage. It is responsible for generating unique integer IDs by tracking the next available ID value. Validation logic is encapsulated here: the validateTitle() method throws an exception when a blank title is provided, enforcing the non-empty title business rule. The cleanDescription() helper trims whitespace from description values.

TaskFileStorage

TaskFileStorage handles all file input/output for task data. It reads from and writes to a plain text file at a Path specified at construction. Each line in the file represents one task record formatted as pipe-delimited fields. To handle edge cases, the class escapes pipe characters, backslashes, and newline characters within field values before saving, and unescapes them when loading. If the file does not exist at load time, an empty list is returned rather than throwing an error.

TaskService

TaskService acts as the application-facing service layer that bridges the GUI to the underlying business logic and persistence. It holds references to both TaskManager and TaskFileStorage. When constructed, it immediately loads tasks from storage and passes them to the TaskManager. Every mutating operation (add, update, complete, delete) is delegated to TaskManager, and if it succeeds, TaskService automatically calls saveTasks() to persist the change.

TodoFrame

TodoFrame is the main application window (JFrame subclass). It displays a JTable populated with task data from TaskService. A JComboBox allows the user to switch between viewing all tasks or only completed tasks. Action buttons trigger the corresponding task operations. The frame opens TaskDialog for add and edit flows, then refreshes the table. Errors from I/O operations are surfaced to the user via JOptionPane dialogs.

TaskDialog

TaskDialog is a modal JDialog that collects task input. It contains a JTextField for the title and a JTextArea for the description. When opened for editing, it prefills both fields from the supplied Task object. Before closing, it validates that the title is not blank. The result is returned as an Optional<TaskFormData>, where TaskFormData is a nested Java record holding the title and description strings. If the user cancels, Optional.empty() is returned.

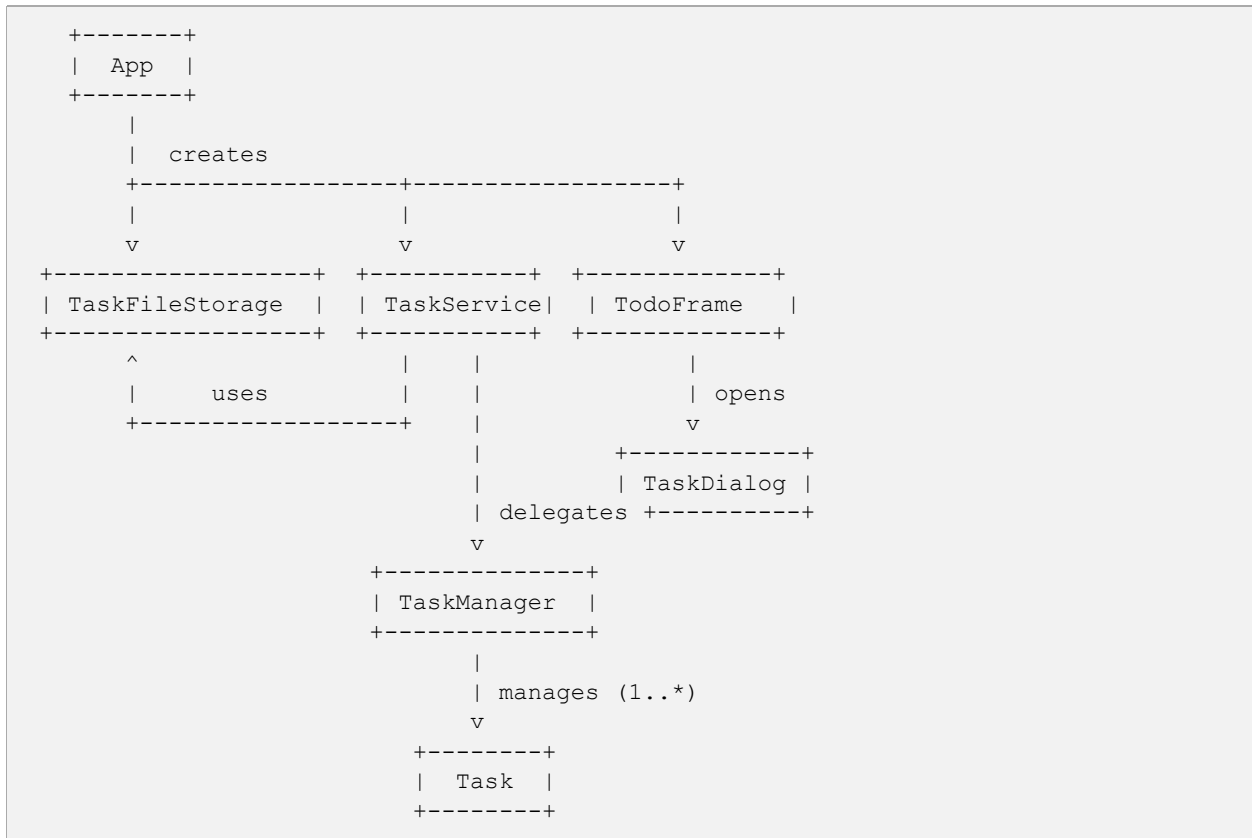
4.2 Business Rules

#	Rule
1	A task must have a non-empty title. Blank titles are rejected by TaskManager.validateTitle().
2	Task descriptions may be blank. An empty description is a valid task record.
3	New tasks are always created with completed = false.
4	Each task receives a unique, system-generated integer ID assigned by TaskManager.
5	When tasks are loaded from file, the next new task ID continues after the highest existing ID in the loaded list.
6	Completed tasks remain visible in the completed tasks filter view.
7	The all-tasks filter view displays both pending and completed tasks.
8	Deleted tasks are removed from the in-memory list and the change is immediately saved to the file.
9	Task data is saved automatically after every successful add, update, mark-complete, or delete operation.
10	If the task file does not exist at startup, the application starts with an empty task list (no error is thrown).
11	File storage must faithfully preserve task title, description, ID, and completion status across save/load cycles.
12	GUI actions for edit, complete, and delete require the user to first select a task in the table.
13	The delete operation displays a confirmation dialog before permanently removing a task.

5. Class Relationship Diagram

5.1 Relationship Overview

The diagram below illustrates how the seven user-defined classes relate to one another. Arrows indicate the direction of dependency or creation.



5.2 Relationship Descriptions

Relationship	Description
App → TaskFileStorage	App creates a TaskFileStorage instance pointing to the default task file path.
App → TaskService	App creates a TaskService, passing the TaskFileStorage to it.
App → TodoFrame	App creates the TodoFrame and passes the TaskService to it.
TodoFrame → TaskService	TodoFrame uses TaskService to perform all task reads and mutations.

TodoFrame → TaskDialog	TodoFrame opens TaskDialog (modal) when the user initiates an add or edit action.
TaskDialog → TaskFormData	TaskDialog creates and returns a TaskFormData record containing the user's input.
TaskService → TaskManager	TaskService delegates all business-logic operations to TaskManager.
TaskService → TaskFileStorage	TaskService calls TaskFileStorage to load tasks at startup and save after mutations.
TaskManager → Task (1..*)	TaskManager manages a collection of Task objects; each TaskManager holds zero or more Tasks.
TaskFileStorage → Task (1..*)	TaskFileStorage serializes and deserializes collections of Task objects from/to file.
TodoFrame → Task (display)	TodoFrame reads Task fields to populate rows in the JTable display.

5.3 Architectural Summary

App is the application entry point. It wires together the other components at startup.

Task is the core domain entity. Every other class either creates, stores, or displays Task objects.

TaskManager owns the task collection and enforces business rules. It has no knowledge of the GUI or file system.

TaskFileStorage handles all file I/O. It translates between Task objects and text-file records. It has no knowledge of the GUI or business logic.

TaskService is the coordination layer. It composes TaskManager and TaskFileStorage, exposing a clean API to the GUI.

TodoFrame is the main GUI window. It consumes TaskService and presents data to the user.

TaskDialog is a focused, reusable modal dialog. It collects task input and returns it to TodoFrame via a typed record.